

CYNA

Modifications réunion tuteur

Récapitulatif des changements

Adrien CACHOUX, Romain CANER, Ethan MIRBEAU

Projet fil rouge - Version 1.2

Solutions de cybersécurité SaaS — SOC · EDR · XDR

Sommaire

Modifications suite à la réunion tuteur

Vue d'ensemble

1. Facture — TVA, remise et détails
2. Abonnements — renouvellement automatique et résiliation
3. Back-office — page « Abonnements »
4. Titre des codes de réduction
5. Support (application Java) — temps réel
6. Facture générée par Stripe (code prêt, à tester)

Ce qu'il reste à faire

Comment relancer le projet (rappel)

Modifications suite à la réunion tuteur

Projet Cyna — Version 1.2 Récapitulatif des changements réalisés après la réunion, pour reprendre au clair.

Toutes ces modifications sont sur la branche Git `feature/facturation-stripe` (elles seront fusionnées dans `main` une fois le paiement Stripe testé avec de vraies clés de test).

Vue d'ensemble

#	Demande du tuteur	État
1	Facture : TVA + remise + plus de détails	[OK] Fait et vérifié
2	Abonnements : renouvellement auto, résiliation correcte	[OK] Fait et vérifié
3	Back-office : page de consultation des abonnements	[OK] Fait
4	Titre / libellé pour les codes de réduction	[OK] Fait et vérifié
5	Support Java : affichage temps réel	[OK] Code présent (nécessite un rebuild)
6	Facture générée par Stripe	[~] Code prêt — à tester avec tes clés
7	E-mails via Mailjet	[...] À faire (nécessite le compte Mailjet)
8	Tests élargis + DCT avec diagrammes UML	[...] À faire (plus tard)

Tests automatisés : 77 PHP + 10 Java, tous au vert.

1. Facture — TVA, remise et détails

La facture (PDF **et** affichage à l'écran) présente désormais un vrai récapitulatif financier :

Sous-total	99,80 €	(si un code promo est appliqué)
Remise (CYNA25)	-24,95 €	
Total HT	62,38 €	
TVA (20 %)	12,47 €	
TOTAL TTC	74,85 €	

- Les prix affichés sont **TTC** ; la **TVA (20 %)** est calculée à rebours ($HT = TTC / 1,20$).
- Ajout des **mentions légales** (SIRET + taux de TVA) en pied de facture.
- Le même récapitulatif apparaît sur la **page de confirmation** et le **détail de commande** (partial réutilisable `order_totals`).

Fichiers : `src/Services/InvoicePdf.php` , `resources/views/partials/order_totals.php` , `resources/views/front/confirmation.php` , `.../account/order_detail.php` .

2. Abonnements — renouvellement automatique et résiliation

C'était le point le plus important de la réunion. Avant, résilier **désactivait immédiatement** l'abonnement, ce qui n'a pas de sens sans notion de renouvellement automatique. C'est corrigé :

- **Renouvellement automatique** : chaque abonnement a un indicateur « se renouvelle seul » (activé par défaut) et une **date d'échéance**.
- **Résilier** = on **désactive le renouvellement automatique**, mais le service **reste actif jusqu'à la date de fin** (l'échéance). C'est ce qu'attendait le tuteur.
- **Réactiver** : on peut ré-enclencher le renouvellement tant que l'abonnement est encore actif.
- À la consultation, les échéances passées sont traitées : renouvellement (si auto) ou passage à « expiré » (si résilié). (En production, une tâche planifiée jouerait ce traitement.)

Affichage côté client : - « Actif · renouvellement le JJ/MM/AAAA » (si auto activé) - « Résilié — actif jusqu'au JJ/MM/AAAA (pas de renouvellement) » (si résilié)

Vérifié en réel : création (échéance à +1 mois), résiliation (reste actif, auto = 0), réactivation (auto = 1).

Fichiers : `src/Repositories/SubscriptionRepository.php` , `src/Controllers/Front/AccountController.php` , `resources/views/front/account/subscriptions.php` , migration `database/migrations/2026_07_abonnements_auto.sql` .

3. Back-office — page « Abonnements »

Le tuteur a noté qu'on n'avait **aucune vue sur les abonnements** côté admin. C'est ajouté : un menu « **Abonnements** » dans le back-office liste tous les abonnements avec **client, service, périodicité, échéance, renouvellement auto, statut**, avec filtre par statut et pagination.

Fichiers : `src/Controllers/Admin/SubscriptionController.php`, `resources/views/admin/subscriptions/index.php`, routes et menu admin.

4. Titre des codes de réduction

Chaque code promo peut désormais avoir un **titre / libellé** lisible (ex : « Offre de bienvenue »), saisi dans le back-office et affiché au panier à côté du code.

Vérifié en réel : « Offre de bienvenue (BIENVENUE10) » s’affiche au panier.

Fichiers : `src/Repositories/PromotionRepository.php`, `src/Services/Promotion.php`, `src/Controllers/Admin/PromotionController.php`, vues panier et admin, migration `database/migrations/2026_07_promo_titre.sql`.

5. Support (application Java) — temps réel

Le rafraîchissement automatique (toutes les 5 s) **est bien dans le code** et compile. Le bug « il faut recharger l’application » venait du fait que **l’ancien** `.exe` (compilé avant les modifications) était lancé.

=> **Il faut reconstruire l’application** pour avoir la version à jour :

```
# depuis le dossier projet-fils-rouge :
docker run --rm -v "${PWD}:/app" -v cyna-m2:/root/.m2 -w /app maven:3.9-eclipse-temurin-17 mvn package
java -jar target/CynaAdminPanel.jar
```

6. Facture générée par Stripe (code prêt, à tester)

Le code est écrit pour que, **quand de vraies clés Stripe sont configurées**, la facture soit **générée par Stripe** (avec la remise en ligne négative), et que le PDF hébergé par Stripe soit servi au téléchargement. **Sans clés**, on retombe automatiquement sur la facture maison (§1).

À faire ce soir

1. Récupérer les **clés de test** sur dashboard.stripe.com (mode test -> Développeurs -> Clés API).
2. Créer le fichier `Cyna/.env` (ignoré par Git) avec :

```
STRIPE_SECRET_KEY=sk_test_XXXXX
STRIPE_PUBLISHABLE_KEY=pk_test_XXXXX
STRIPE_CURRENCY=eur
```

3. Appliquer la migration facture Stripe (ou repartir propre) :

```
docker compose down -v
docker compose up -d --build
```

4. Passer une commande avec la carte de test **4242 4242 4242 4242** + un code promo, puis télécharger la facture -> elle doit venir de **Stripe**.

Fichiers : `src/Services/PaymentService.php` (`createInvoice`), `src/Controllers/Front/CheckoutController.php` , `../AccountController.php` , migration `database/migrations/2026_07_stripe_invoice.sql` .

Ce qu'il reste à faire

- **Stripe (ce soir)** : ajouter les clés, tester le paiement réel + la facture Stripe, puis **fusionner la branche dans `main`** .
- **Mailjet** : brancher l'envoi réel d'e-mails (nécessite le compte Mailjet).
- **Tests** : élargir la couverture (le tuteur veut « tous les cas pertinents »).
- **DCT** : ajouter des diagrammes UML (cas d'utilisation, séquence, classes).

Comment relancer le projet (rappel)

```
cd projet-fils-rouge/Cyna
git checkout feature/facturation-stripe # la branche de ces modifications
docker compose down -v
docker compose up -d --build
# Site : http://localhost:8080 | E-mails de test : http://localhost:8025
# Admin : admin@cyna-it.fr / Admin@1234 | Client : client@cyna-it.fr /
Client@1234
```